

CENTRO UNIVERSITÁRIO FEI
MATHEUS BERNARDINELI SEGURA
R.A.: 24.125.036-4

PROJETO FEITV

Relatório Final do Projeto apresentado ao
Centro Universitário FEI, como parte dos
requisitos da Disciplina “CCP230 –
Desenvolvimento de Algoritmos” ministrada
pela Prof^ª. Dra. Gabriela Biondi

São Paulo
2026
SUMÁRIO

1	INTRODUÇÃO	1
2	DESENVOLVIMENTO	2
3	RESULTADOS	3
4	CONCLUSÕES	4
5	REFERÊNCIAS	5

1 INTRODUÇÃO

O projeto FEITV tem como objetivo desenvolver uma plataforma digital de streaming inspirada em serviços de vídeo online, permitindo que usuários realizem cadastro, login, busca de vídeos, curtidas e gerenciamento de playlists.

O sistema foi desenvolvido utilizando a linguagem C, com persistência de dados feita através de arquivos texto (.txt). Essa abordagem permitiu implementar funcionalidades completas de armazenamento, utilizando conceitos de manipulação de arquivos, structs, modularização e tratamento de strings.

O projeto foi feito individualmente e contempla funcionalidades como:

- Cadastro de usuários;
- Sistema de login;
- Busca de vídeos;
- Curtir e descurtir vídeos;
- Criação de playlists;
- Adição e remoção de vídeos em playlists;
- Exclusão de playlists;
- Persistência de dados em arquivos texto.

O sistema foi organizado em arquivos .c e .h, facilitando a manutenção do código e a separação das funcionalidades de cada módulo.

2 DESENVOLVIMENTO

O sistema FEITV foi desenvolvido utilizando arquitetura modular, separando as funcionalidades em diferentes arquivos.

□ Estrutura do Projeto:

O projeto foi dividido da seguinte forma:

- main.c: menus e navegação do sistema;
- usuarios.c: cadastro e login de usuários;
- videos.c: busca, curtidas e descurtidas;
- playlist.c: gerenciamento de playlists;
- feitv.h: structs e protótipos das funções.

□ Estrutura dos Arquivos de Dados:

As informações do sistema são armazenadas em arquivos .txt.

usuarios.txt: Armazena os usuários cadastrados no formato:

id email senha nome



usuarios.txt - Bloco de Notas

```
Arquivo  Editar  Formatar  Exibir  Ajuda
1 matheus@email.com 1234 Matheus
2 mario@email.com 5678 Mario
3 luigi@email.com 1225 Luigi
4 carlos@email.com 2512 Carlos
5 marcos@email.com 4567 Marcos
```

videos.txt: Armazena os vídeos cadastrados no sistema:

id titulo categoria tipo duracao temporadas episodios likes



videos.txt - Bloco de Notas

```
Arquivo  Editar  Formatar  Exibir  Ajuda
1 Vingadores Acao F 150 0 0 230
2 Titanic Romance F 180 0 0 191
3 Batman Acao F 140 0 0 300
4 Avatar Ficcao F 162 0 0 220
5 Titanic_2 Romance F 194 0 0 170
6 Matrix Ficcao F 136 0 0 250
7 Gladiador Acao F 155 0 0 189
8 Barbie Comedia F 114 0 0 130
9 Oppenheimer Drama F 180 0 0 210
10 Batman_O_Cavaleiro Acao F 176 0 0 300
11 Frozen Animacao F 102 0 0 160
12 Up_Altas_Aventuras Animacao F 96 0 0 140
13 Deadpool Acao F 108 0 0 280
14 Homem_Aranha Acao F 121 0 0 260
15 Parasita Suspense F 132 0 0 175
```

```

16 Breaking_Bad Drama S 0 5 62 500
17 Friends Comedia S 0 10 236 700
18 Dark Suspense S 0 3 26 400
19 Stranger_Things Ficcao S 0 4 34 450
20 The_Boys Acao S 0 3 24 410
21 Loki Ficcao S 0 2 12 370
22 Wandinha Suspense S 0 1 8 340
23 Arcane Animacao S 0 1 9 460
24 The_Office Comedia S 0 9 201 380
25 Round_6 Suspense S 0 1 9 430
26 Naruto Anime S 0 9 220 520
27 One_Piece Anime S 0 20 1000 600
28 Jujutsu_Kaisen Anime S 0 2 47 490
29 The_Last_of_Us Drama S 0 1 9 470
30 House_of_the_Dragon Fantasia S 0 1 10 440

```

playlist.txt: Armazena as playlists dos usuários:

id_playlist nome_playlist id_usuario {ids_videos}



playlist.txt - Bloco de Notas

```

Arquivo  Editar  Formatar  Exibir  Ajuda
1 Mario_Playlist 2 {1,2,3}
2 Matheus_Playlist 1 {4,5,6}

```

curtidas.txt: Armazena quais usuários curtiram determinados vídeos:

idUsuario idVideo



curtidas.txt - Bloco de Notas

```

Arquivo  Editar  Formatar  Exibir  Ajuda
1 16
1 8
5 2

```

☐ Funcionalidades Implementadas:

Cadastro de Usuário

O usuário pode criar uma nova conta informando:

- ID;
- Nome;
- Email;
- Senha.

O sistema verifica se o ID já existe antes de cadastrar.

Exemplo de funcionamento:

```
--- FEITV: Bem-vindo ---
1 - Cadastrar usuario
2 - Efetuar Login
0 - Sair
Escolha uma opcao: 1

--- Cadastro de Novo Usuario ---
ID do usuario: 1
ID ja cadastrado.
ID do usuario:
```

Login de Usuário

O login é realizado utilizando email e senha.

O sistema percorre o arquivo usuarios.txt verificando se os dados informados correspondem aos cadastrados.



```
feity.h  usuarios.c  main.c  playlist.c  videos.c

46
47 int realizarLogin() {
48     FILE *arq = fopen("usuarios.txt", "r");
49
50     int idArquivo;
51     char emailArquivo[100];
52     char senhaArquivo[30];
53     char nomeArquivo[50];
54
55     char emailInformado[100];
56     char senhaInformada[30];
57
58     if (arq == NULL) {
59         printf("Nenhum usuario cadastrado.\n");
60         return 0;
61     }
62
63     printf("Email: ");
64     scanf("%s", emailInformado);
65
66     printf("Senha: ");
67     scanf("%s", senhaInformada);
68
69     while (fscanf(arq, "%d %s %s %s",
70                 &idArquivo,
71                 emailArquivo,
72                 senhaArquivo,
73                 nomeArquivo) == 4) {
74
75         if (strcmp(emailInformado, emailArquivo) == 0 &&
76             strcmp(senhaInformada, senhaArquivo) == 0) {
77
78             printf("Bem-vindo, %s!\n", nomeArquivo);
79             fclose(arq);
80
81             return idArquivo;
82         }
83     }
84
85     fclose(arq);
86     printf("Email ou senha incorretos.\n");
87     return 0;
88 }
89
```

Busca de Vídeos

A funcionalidade permite buscar vídeos pelo título.

São exibidas informações como:

- ID;
- Título;
- Categoria;
- Tipo (Filme ou Série);
- Duração;
- Temporadas;
- Episódios;
- Likes.

```
--- FEITV: Menu do usuario ---
1 - Buscar Video
2 - Curtir Video
3 - Descurtir Video
4 - Criar Playlist
5 - Ver Minhas Playlists
6 - Adicionar Video
7 - Remover Video
8 - Excluir Playlist
9 - Logout
Opcao: 1
Digite o nome: Breaking_Bad

--- Resultados de Busca ---
ID: 16
Titulo: Breaking_Bad
Categoria: Drama
Tipo: Serie
Temporadas: 5
Episodios: 62
Likes: 500
Pressione qualquer tecla para continuar. . .
```

```
Opcao: 1
Digite o nome: Batman

--- Resultados de Busca ---
ID: 3
Titulo: Batman
Categoria: Acao
Tipo: Filme
Duracao: 140 min
Likes: 300
Pressione qualquer tecla para continuar. . .
```

Curtir Vídeos

O usuário pode curtir vídeos.

O sistema:

- Verifica se o vídeo existe;
- Verifica se o usuário já curtiu;
- Atualiza o número de likes em videos.txt;
- Registra a curtida em curtidas.txt.

Exemplo de funcionamento:

```
feity.h  usuarios.c  main.c  playlist.c  videos.c

55 void curtirVideo(int idUsuarioLogado, int idProcurado) {
56     FILE *arqCurtidas = fopen("curtidas.txt", "a+");
57     int usuariolido, videolido;
58
59     if (arqCurtidas == NULL) {
60         printf("Erro ao abrir arquivo de curtidas.\n");
61         return;
62     }
63
64     if (!videoExiste(idProcurado)) {
65         printf("Video nao encontrado.\n");
66         fclose(arqCurtidas);
67         return;
68     }
69
70     // verifica se ja curtiu
71     while (fscanf(arqCurtidas, "%d %d",
72                 &usuariolido,
73                 &videolido) == 2) {
74
75         if (usuariolido == idUsuarioLogado &&
76             videolido == idProcurado) {
77
78             printf("Voce ja curtiu esse video.\n");
79             fclose(arqCurtidas);
80             return;
81         }
82     }
83
84     // curtida
85     fprintf(arqCurtidas, "%d %d\n",
86            idUsuarioLogado,
87            idProcurado);
88
89     fclose(arqCurtidas);
90
91     // atualiza likes
92     FILE *arq = fopen("videos.txt", "r");
93
94     Video catalogo[100];
95     int i = 0, total = 0;
96
97     if (arq == NULL) {
98         printf("Arquivo nao encontrado.\n");
99         return;
100     }
101
102     while(fscanf(arq, "%d %s %s %c %d %d %d %d",
103                 &catalogo[i].id,
104                 catalogo[i].titulo,
105                 catalogo[i].categoria,
106                 &catalogo[i].tipo,
107                 &catalogo[i].duracao,
108                 &catalogo[i].temporadas,
109                 &catalogo[i].episodios,
110                 &catalogo[i].likes) == 8) {
111
112         if (catalogo[i].id == idProcurado) {
113             catalogo[i].likes++;
114         }
115
116         i++;
117     }
118
119     total = i;
120     fclose(arq);
121 }
```

```

122     arq = fopen("videos.txt", "w");
123
124     for (i = 0; i < total; i++) {
125         fprintf(arq, "%d %s %s %c %d %d %d %d\n",
126             catalogo[i].id,
127             catalogo[i].titulo,
128             catalogo[i].categoria,
129             catalogo[i].tipo,
130             catalogo[i].duracao,
131             catalogo[i].temporadas,
132             catalogo[i].episodios,
133             catalogo[i].likes);
134     }
135
136     fclose(arq);
137
138     printf("Like atualizado com sucesso!\n");
139 }
140

```

Descurtir Vídeos

O sistema remove a curtida do usuário e decrementa os likes do vídeo.

Também utiliza um arquivo temporário para reescrever os dados atualizados.

Exemplo:

```

feittv.h  usuarios.c  main.c  playlist.c  videos.c
141 void descurtirVideo(int idUsuarioLogado, int idProcurado) {
142     FILE *arqCurtidas = fopen("curtidas.txt", "r");
143     FILE *temp = fopen("temp_curtidas.txt", "w");
144
145     int usuarioLido, videoLido;
146     int encontrouCurtida = 0;
147
148     if (arqCurtidas == NULL || temp == NULL) {
149         printf("Erro ao abrir arquivo de curtidas.\n");
150         return;
151     }
152
153     // remove curtida do usuario
154     while (fscanf(arqCurtidas, "%d %d",
155         &usuarioLido,
156         &videoLido) == 2) {
157
158         if (usuarioLido == idUsuarioLogado &&
159             videoLido == idProcurado) {
160
161             encontrouCurtida = 1; // nao ggrava no temp = remove
162
163         } else {
164             fprintf(temp, "%d %d\n",
165                 usuarioLido,
166                 videoLido);
167         }
168     }
169
170     fclose(arqCurtidas);
171     fclose(temp);
172
173     remove("curtidas.txt");
174     rename("temp_curtidas.txt", "curtidas.txt");
175
176     if (!encontrouCurtida) {
177         printf("Voce ainda nao curtiu esse video.\n");
178         return;
179     }
180
181     // diminui o like
182     FILE *arq = fopen("videos.txt", "r");
183
184     Video catalogo[100];
185     int i = 0, total = 0;

```



```

187 | if (arq == NULL) {
188 |     printf("Arquivo nao encontrado.\n");
189 |     return;
190 | }
191 |
192 | while (fscanf(arq, "%d %s %s %c %d %d %d %d",
193 |             &catalogo[i].id,
194 |             catalogo[i].titulo,
195 |             catalogo[i].categoria,
196 |             &catalogo[i].tipo,
197 |             &catalogo[i].duracao,
198 |             &catalogo[i].temporadas,
199 |             &catalogo[i].episodios,
200 |             &catalogo[i].likes) == 8) {
201 |     if (catalogo[i].id == idProcurado &&
202 |         catalogo[i].likes > 0) {
203 |         catalogo[i].likes--;
204 |     }
205 |
206 |     i++;
207 | }
208 |
209 | total = i;
210 | fclose(arq);
211 |
212 | arq = fopen("videos.txt", "w");
213 |
214 | for (i = 0; i < total; i++) {
215 |     fprintf(arq, "%d %s %s %c %d %d %d %d\n",
216 |             catalogo[i].id,
217 |             catalogo[i].titulo,
218 |             catalogo[i].categoria,
219 |             catalogo[i].tipo,
220 |             catalogo[i].duracao,
221 |             catalogo[i].temporadas,
222 |             catalogo[i].episodios,
223 |             catalogo[i].likes);
224 | }
225 |
226 | fclose(arq);
227 |
228 | printf("Video descurtido com sucesso!\n");
229 | }
230 |
231 |

```

Criação de Playlists

O usuário pode criar playlists personalizadas.

Cada playlist possui:

- ID;
- Nome;
- Dono da playlist;
- Lista de IDs de vídeos.

Exemplo:

```

Opcao: 5

--- Minhas Playlists ---
ID: 2
Nome: Matheus_Playlist
Videos: {4,5,6}
4 - Avatar, Filme
5 - Titanic_2, Filme
6 - Matrix, Filme
-----
Pressione qualquer tecla para continuar. . .

```

Adicionar Vídeos à Playlist

O sistema permite adicionar vídeos a playlists já existentes.

São realizadas validações como:

- Verificar se a playlist pertence ao usuário;
- Verificar se o vídeo existe;
- Verificar se o vídeo já está na playlist.

Exemplo: (foto)

```
feitr.h  usuarios.c  main.c  playlist.c  videos.c

296
297 void adicionarVideoPlaylist(int idPlaylistAlvo, int idNovoVideo, int idUsuarioLogado)
298     FILE *arq = fopen("playlist.txt", "r");
299
300     Playlist playlists[100];
301     int i = 0;
302     int total = 0;
303     int encontrada = 0;
304
305     char novoIdStr[20];
306
307     if (arq == NULL) {
308         printf("Arquivo de playlist nao encontrado.\n");
309         return;
310     }
311
312     if (!videoExiste(idNovoVideo)) {
313         printf("Video nao encontrado.\n");
314         fclose(arq);
315         return;
316     }
317
318 // lendo todas as playlists do arq (aula 08)
319 while (fscanf(arq, "%d %s %d %s",
320             &playlists[i].id_p,
321             playlists[i].nome_p,
322             &playlists[i].id_u,
323             playlists[i].lista_videos) != EOF) {
324     //se for playlist alvo, modifica a string (aula 07)
325     if (playlists[i].id_p == idPlaylistAlvo &&
326         playlists[i].id_u == idUsuarioLogado) {
327         char copia[100];
328         strcpy(copia, playlists[i].lista_videos);
329
330         copia[strlen(copia) - 1] = '\0';
331
332         char *token = strtok(copia + 1, ",");
333
334         while (token != NULL) {
335             if (atoi(token) == idNovoVideo) {
336                 printf("Esse video ja esta na playlist.\n");
337                 encontrada = -1;
338                 break;
339             }
340             token = strtok(NULL, ",");
341         }
342
343         if (encontrada == -1) {
344             break;
345         }
346
347         // remove '}' para concatenar
348         int tam = strlen(playlists[i].lista_videos); // calcula comprimento
349
350         if (tam > 0 && playlists[i].lista_videos[tam - 1] == '}') {
351             playlists[i].lista_videos[tam - 1] = '\0'; // corta o fecha chaves
352         }
353
354         // formata o novo id e anexa usando strcat
355         sprintf(novoIdStr, "%d", idNovoVideo);
356         strcat(playlists[i].lista_videos, novoIdStr);
357
358         encontrada = 1;
359     }
360 }
361
```

```

361 |         i++;
362 |     }
363 | }
364 |
365 | total = i;
366 | fclose(arq);
367 |
368 | if (encontrada == -1) {
369 |     return;
370 | }
371 |
372 | if (encontrada == 0) {
373 |     // sobrescreve arq com dados atualizados
374 |     printf("Playlist nao encontrada ou nao pertence a voce.\n");
375 |     return;
376 | }
377 |
378 | arq = fopen("playlist.txt", "w");
379 |
380 | for (i = 0; i < total; i++) {
381 |     fprintf(arq, "%d %s %d %s\n",
382 |             playlists[i].id_p,
383 |             playlists[i].nome_p,
384 |             playlists[i].id_u,
385 |             playlists[i].lista_videos);
386 | }
387 |
388 | fclose(arq);
389 |
390 | printf("Video %d adicionado com sucesso!\n", idNovoVideo);
391 | }

```

Remover Vídeos da Playlist

O usuário pode remover vídeos específicos de uma playlist.

O sistema percorre a string contendo os IDs dos vídeos e recria a lista sem o vídeo removido.

Exemplo:

```

feitv.h  usuarios.c  main.c  playlist.c  videos.c
4  #include "feitv.h"
5
6  void removerVideoPlaylist(int idPlaylistAlvo, int idVideoRemover, int idUsuarioLogado) {
7      FILE *arq = fopen("playlist.txt", "r");
8
9      Playlist playlists[100];
10     int i = 0;
11     int total = 0;
12     int encontrada = 0;
13     int removeu = 0;
14
15     if (arq == NULL) {
16         printf("Arquivo de playlist nao encontrado.\n");
17         return;
18     }
19
20     // le as playlists
21     while (fscanf(arq, "%d %s %d %s",
22                 &playlists[i].id_p,
23                 playlists[i].nome_p,
24                 &playlists[i].id_u,
25                 playlists[i].lista_videos) != EOF) {
26         i++;
27     }
28
29     total = i;
30     fclose(arq);
31
32     // procura playlist
33     for (i = 0; i < total; i++) {
34         if (playlists[i].id_p == idPlaylistAlvo &&
35             playlists[i].id_u == idUsuarioLogado) {
36             encontrada = 1;
37
38             char novaLista[100] = "{";
39             char copia[100];
40             int primeiro = 1;
41
42             strcpy(copia, playlists[i].lista_videos);
43
44             // remove { e }
45             copia[strlen(copia) - 1] = '\0';
46
47             char *token = strtok(copia + 1, ",");
48

```

```

48
49
50     while (token != NULL) {
51         int idAtual = atoi(token);
52
53         if (idAtual != idVideoRemover) {
54
55             if (!primeiro) {
56                 strcat(novaLista, ",");
57             }
58
59             strcat(novaLista, token);
60             primeiro = 0;
61         } else {
62             removeu = 1;
63         }
64
65         token = strtok(NULL, ",");
66     }
67
68     strcat(novaLista, "\n");
69     strcpy(playlists[i].lista_videos, novaLista);
70 }
71
72 if (!encontrada) {
73     printf("Playlist nao encontrada ou nao pertence a voce.\n");
74     return;
75 }
76
77 if (!removeu) {
78     printf("Esse video nao existe na playlist.\n");
79     return;
80 }
81
82 // regrava arquivo
83 arq = fopen("playlist.txt", "w");
84
85 for (i = 0; i < total; i++) {
86     fprintf(arq, "%d %s %d %s\n",
87         playlists[i].id_p,
88         playlists[i].nome_p,
89         playlists[i].id_u,
90         playlists[i].lista_videos);
91 }
92
93
94
95     fclose(arq);
96     printf("Video removido da playlist com sucesso!\n");
97 }

```

Exclusão de Playlists

O sistema permite excluir playlists utilizando um arquivo temporário.

A playlist só pode ser removida pelo próprio dono.

Exemplo:

feittv.h	usuarios.c	main.c	playlist.c	videos.c
98			void excluirPlaylist(int idPlaylistExcluir, int idUsuarioLogado) {	
99			FILE *arq = fopen("playlist.txt", "r");	
100			FILE *temp = fopen("temp.txt", "w");	
101				
102			Playlist p;	
103			int achou = 0;	
104			int encontrouPlaylist = 0;	
105				
106			if (arq == NULL temp == NULL) {	
107			printf("Erro ao abrir banco de dados!\n");	
108			return;	
109			}	
110				

```

111 while (fscanf(arq, "%d %s %d %s",
112             &p.id_p,
113             p.nome_p,
114             &p.id_u,
115             p.lista_videos) == 4) {
116     if (p.id_p == idPlaylistExcluir) {
117         encontrouPlaylist = 1;
118         if (p.id_u == idUsuarioLogado) {
119             achou = 1; // exclui
120         } else {
121             printf("Voce nao pode excluir a playlist de outro usuario.\n");
122             fprintf(temp, "%d %s %d %s\n",
123                     p.id_p,
124                     p.nome_p,
125                     p.id_u,
126                     p.lista_videos);
127         }
128     } else {
129         fprintf(temp, "%d %s %d %s\n",
130                 p.id_p,
131                 p.nome_p,
132                 p.id_u,
133                 p.lista_videos);
134     }
135 }
136 fclose(arq);
137 fclose(temp);
138 remove("playlist.txt");
139 rename("temp.txt", "playlist.txt");
140
141 if (!encontrouPlaylist) {
142     printf("Playlist nao encontrada.\n");
143 }
144 else if (achou) {
145     printf("Playlist excluida com sucesso!\n");
146 }
147 }

```

3 RESULTADOS

Com base nos testes realizados, todas as funcionalidades implementadas apresentaram funcionamento correto.

Foi possível:

- Cadastrar usuários;
- Realizar login;
- Buscar vídeos;
- Curtir e descurtir vídeos;
- Criar e excluir playlists;
- Adicionar e remover vídeos;
- Persistir os dados corretamente em arquivos textos.

□ Demonstração da Persistência de Dados:

Antes do cadastro

Arquivo Editar Formatar Exibir Ajuda

1 matheus@email.com 1234 Matheus
2 mario@email.com 5678 Mario
3 luigi@email.com 1225 Luigi
4 carlos@email.com 2512 Carlos
|

Após o cadastro

Arquivo Editar Formatar Exibir Ajuda

1 matheus@email.com 1234 Matheus
2 mario@email.com 5678 Mario
3 luigi@email.com 1225 Luigi
4 carlos@email.com 2512 Carlos
5 marcos@email.com 4567 Marcos

Antes da curtida

Arquivo Editar Formatar Exibir Ajuda

1 16
1 8
|

Após a curtida

Arquivo Editar Formatar Exibir Ajuda

1 16
1 8
5 2

Antes da criação da playlist

Arquivo Editar Formatar Exibir Ajuda

```
1 Mario_Playlist 2 {1,2,3}
2 Matheus_Playlist 1 {4,5,6}
```

Após criar playlist

Arquivo Editar Formatar Exibir Ajuda

```
1 Mario_Playlist 2 {1,2,3}
2 Matheus_Playlist 1 {4,5,6}
3 Marcos_Playlist 5 {1,2,3}
```

☐ Validação das Regras

O sistema também trata operações inválidas.

Curtida duplicada

```
--- FEITV: Menu do usuario ---
1 - Buscar Video
2 - Curtir Video
3 - Descurtir Video
4 - Criar Playlist
5 - Ver Minhas Playlists
6 - Adicionar Video
7 - Remover Video
8 - Excluir Playlist
9 - Logout
Opcao: 2
ID do video: 16
Voce ja curtiu esse video.
Pressione qualquer tecla para continuar. . .
```

Playlist de outro usuário

```
8 - Excluir Playlist
9 - Logout
Opcao: 8
ID da playlist: 3
Voce nao pode excluir a playlist de outro usuario.
Pressione qualquer tecla para continuar. . .
```

Vídeo inexistente

```
--- FEITV: Menu do usuario ---
1 - Buscar Video
2 - Curtir Video
3 - Descurtir Video
4 - Criar Playlist
5 - Ver Minhas Playlists
6 - Adicionar Video
7 - Remover Video
8 - Excluir Playlist
9 - Logout
Opcao: 1
Digite o nome: a

--- Resultados de Busca ---
Nenhum video encontrado.
Pressione qualquer tecla para continuar. . .
```

4 CONCLUSÕES

O projeto FEITV atingiu o objetivo proposto, implementando um sistema funcional de streaming utilizando linguagem C e persistência de dados através de arquivos texto.

Durante o desenvolvimento foram aplicados conceitos de programação como:

- Modularização;
- Structs;
- Manipulação de arquivos;
- Manipulação de strings;
- Vetores;
- Funções;
- Validação de dados;
- Reescrita de arquivos.

Mesmo sendo um sistema simples, o projeto possui potencial para futuras melhorias como:

- Banco de dados;
- Cadastro administrativo;
- Recomendações de vídeos;
- Sistema de comentários;
- Busca parcial de títulos.

O desenvolvimento do projeto permitiu aprofundar conhecimentos na linguagem C e compreender melhor o funcionamento interna de sistemas de persistência de dados.

Durante o desenvolvimento foram aplicados conceitos de programação como:

- Modularização;
- Structs;
- Manipulação de arquivos;
- Manipulação de strings;
- Vetores;
- Funções;
- Validação de dados;
- Reescrita de arquivos.

Mesmo sendo um sistema simples, o projeto possui potencial para futuras melhorias como:

- Banco de dados;
- Cadastro administrativo;
- Recomendações de vídeos;
- Sistema de comentários;
- Busca parcial de títulos.

5 REFERÊNCIAS

<https://www.youtube.com/watch?v=CKEbTMko0nM>

https://www.youtube.com/watch?v=Fom_aOypAqg

<https://www.youtube.com/watch?v=yKoLERQUZ1A>

<https://www.ibm.com/docs/pt-br/i/7.5.0?topic=functions-atoi-convert-character-string-integer>

<https://linguagemc.com.br/argumentos-em-linha-de-comando/>

<https://pt.stackoverflow.com/questions/182074/polimorfismo-em-c>